

QR DECOMPOSITION

Using Gram-Schmidt with Column Operations in Java

A Technical Framework

LinSolve

Version 1.0

2026

LinSolve

Contents

1	Introduction	1
1.1	Why Decompose Matrices?	1
1.2	What is QR Decomposition?	1
1.2.1	Visual Representation	1
1.2.2	Why QR is Important	2
1.3	Overview of the Elementary Matrix Approach	2
1.3.1	The Elementary Matrix Philosophy	2
1.3.2	The Two Elementary Operations for QR	2
1.3.3	What This Framework Covers	2
1.4	Prerequisites	2
1.5	Guide Structure	3
1.6	Notation Conventions	3
1.7	Getting Started	3
2	Core Matrix Components	5
2.1	Introduction	5
2.2	The Matrix Class	5
2.2.1	Class Implementation	5
2.3	The SquareMatrix Class	7
2.3.1	Class Implementation	7
2.4	The Identity Class	7
2.4.1	Class Implementation	8
3	The Eij Component - Basis of Matrix Vector Space	9
3.1	Introduction	9
3.2	What is Eij?	9
3.2.1	Examples of Eij	9
3.2.2	Class Implementation	9
3.3	Elementary Matrix Construction Using Eij	10
4	Basic Matrix Operations	11
4.1	Matrix Addition	11
4.2	Matrix Multiplication	11
4.3	Scalar Multiplication	12
4.4	Matrix Transpose	12

5	Elementary Column Operations	15
5.1	Introduction	15
5.2	Column Addition: $C_i \leftarrow C_i + \lambda C_j$	15
5.2.1	Example	15
5.2.2	Java Implementation	15
5.3	Column Scaling: $C_i \leftarrow \lambda C_i$	17
5.3.1	Example	17
5.3.2	Java Implementation	17
5.4	Why These Operations are Invertible	18
6	Gram-Schmidt Algorithm	19
6.1	Introduction	19
6.2	Mathematical Formulation	19
6.2.1	Orthogonalization Step	19
6.2.2	Normalization Step	19
6.3	Column Operation Formulation	19
7	QR Decomposition Implementation	21
7.1	The ApplyingGramSchmidtProcess Class	21
7.1.1	Class Implementation	21
7.2	Step-by-Step Example	24
7.2.1	Step 1: Column 0 ($j = 0$)	24
7.2.2	Step 2: Column 1 ($j = 1$)	24
7.2.3	Step 3: Column 2 ($j = 2$)	24
7.2.4	Result	25
8	Verification and Testing	27
8.1	Orthogonality Test	27
8.2	Reconstruction Test	27
8.3	Upper Triangular Test	28
8.4	Test Results	28
8.5	Verification Summary	29
9	Complexity Analysis	31
9.1	Time Complexity	31
9.2	Space Complexity	31
10	Numerical Stability	33
10.1	Stability Analysis	33
10.2	Modified Gram-Schmidt	33
10.3	Tolerance Guidelines	33
A	Complete Code Listing	35
A.1	Generating Random Matrix	35
B	Glossary	37

Listings

2.1	Matrix.java	6
2.2	SquareMatrix.java	7
2.3	Identity.java	8
3.1	Eij.java	9
4.1	AddingMatrices.java	11
4.2	MultiplyingMatrices.java	11
4.3	MultiplyingMatrixByScalar.java	12
4.4	TransposingMatrix.java	13
5.1	Ci_plus_lamda_Cj.java	16
5.2	Ci_lamda_Ci.java	17
7.1	ApplyingGramSchmidtProcess.java	21
8.1	Q Matrix Output	28
8.2	R Matrix Output	28
8.3	QR Reconstruction	28
A.1	GeneratingRandomMatrix.java	35

Chapter 1

Introduction

1.1 Why Decompose Matrices?

Solving systems of linear equations is at the heart of scientific computing, machine learning, and engineering. Consider the canonical problem:

$$A\mathbf{x} = \mathbf{b} \quad (1.1)$$

Where A is an $m \times n$ matrix, $\mathbf{x}$ is the unknown vector, and $\mathbf{b}$ is the known right-hand side.

The QR decomposition is one of the most important matrix factorizations in numerical linear algebra.

1.2 What is QR Decomposition?

QR decomposition factors a matrix $A \in \mathbb{R}^{m \times n}$ (with $m \geq n$) into:

$$\boxed{A = Q \times R} \quad (1.2)$$

Where:

- $Q \in \mathbb{R}^{m \times n}$ has **orthonormal columns** ($Q^T Q = I_n$)
- $R \in \mathbb{R}^{n \times n}$ is **upper triangular**

1.2.1 Visual Representation

For a 4×4 matrix:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{21} & q_{22} & q_{23} & q_{24} \\ q_{31} & q_{32} & q_{33} & q_{34} \\ q_{41} & q_{42} & q_{43} & q_{44} \end{bmatrix} \times \begin{bmatrix} r_{11} & r_{12} & r_{13} & r_{14} \\ 0 & r_{22} & r_{23} & r_{24} \\ 0 & 0 & r_{33} & r_{34} \\ 0 & 0 & 0 & r_{44} \end{bmatrix}$$

1.2.2 Why QR is Important

1. **Linear Least Squares:** QR provides a numerically stable solution to $\min \|Ax - b\|_2$
2. **Eigenvalue Computation:** The QR algorithm finds all eigenvalues of a matrix
3. **Orthogonal Basis:** Q provides an orthonormal basis for the column space of A
4. **Stability:** QR is more stable than normal equations for least squares

1.3 Overview of the Elementary Matrix Approach

Traditional QR implementations use nested loops with scalar operations. **This hides the linear algebra.**

1.3.1 The Elementary Matrix Philosophy

Our approach treats **every operation as a matrix multiplication**:

$$A \times E \rightarrow A_{\text{new}} \tag{1.3}$$

Where E is an **elementary matrix** — the identity matrix plus a single modification.

1.3.2 The Two Elementary Operations for QR

1. **Column scaling:** Ci_lamda_Ci — multiply column i by λ
2. **Column addition:** $Ci_plus_lamda_Cj$ — add $\lambda \times$ column j to column i

1.3.3 What This Framework Covers

- Complete implementation of matrix and elementary matrix classes in Java
- Step-by-step construction of QR decomposition using column operations
- Gram-Schmidt orthogonalization process
- Extraction of Q and R from accumulated elementary matrices
- Complete source code project included with this framework

1.4 Prerequisites

- Basic Java programming (classes, loops, arrays)
- Basic linear algebra (matrix multiplication, orthogonality, triangular matrices)
- No external libraries required — everything implemented from scratch

1.5 Guide Structure

Part	Content
Part I	Foundations — Matrix, SquareMatrix, Identity, Eij
Part II	Basic Matrix Operations — Addition, multiplication, scalar multiplication
Part III	Elementary Matrices — Ci_lamda_Ci, Ci_plus_lamda_Cj
Part IV	QR Decomposition — Gram-Schmidt process, column operations
Part V	Complete Implementation — Extracting Q and R
Appendices	Complete source code, mathematical prerequisites, glossary

1.6 Notation Conventions

Throughout this document, we use the following conventions:

- Matrices are denoted by capital letters: A , Q , R
- Vectors are denoted by bold lowercase letters: $\mathbf{x}$, $\mathbf{b}$
- Scalars are denoted by lowercase letters: λ , a_{ij}
- Indices start at 0 (Java convention) for both mathematical formulas and code
- I_n denotes the $n \times n$ identity matrix
- E_{ij} denotes the basis matrix with 1 at position (i, j) and 0 elsewhere

1.7 Getting Started

The complete source code accompanying this framework is organized in the following package structure:

```
com.linsolve.decompositions.qr/
|-- components/
|   |-- Matrix.java
|   |-- SquareMatrix.java
|   |-- Identity.java
|   |-- Eij.java
|-- components.basicoperations/
|   |-- AddingMatrices.java
|   |-- MultiplyingMatrices.java
|   |-- MultiplyingMatrixByScalar.java
|   `-- TransposingMatrix.java
```

```
|-- components.elementaryoperations/  
|   |-- Ci_lamda_Ci.java  
|   `-- Ci_plus_lamda_Cj.java  
|-- components.helper/  
|   |-- GeneratingRandomMatrix.java  
|   `-- InvertingUpperMatrix.java  
`-- operator/  
    `-- ApplyingGramSchmidtProcess.java
```

Chapter 2

Core Matrix Components

2.1 Introduction

The foundation of any matrix computation library is a robust representation of matrices. This chapter presents the core classes that form the building blocks of our QR decomposition framework.

All matrix classes store data in a two-dimensional array of `Double` objects, allowing for:

- Dynamic sizing at runtime
- Null value representation (if needed)
- Easy integration with other numerical methods

2.2 The Matrix Class

The `Matrix` class is the fundamental building block. It stores a rectangular array of `Double` values with dimensions $m \times n$.

Definition

A **Matrix** of size $m \times n$ is a rectangular array of numbers arranged in m rows and n columns:

$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix}$$

The entry in row i and column j is denoted a_{ij} .

2.2.1 Class Implementation

```
1 package com.linsolve.decompositions.qr.components;
2
3 public class Matrix {
4     protected Integer m;          // Number of rows
5     protected Integer n;          // Number of columns
6     protected Double[][] t;       // The data array
7
8     public Matrix() {
9         super();
10    }
11
12    public Matrix(Integer m, Integer n) {
13        super();
14        this.m = m;
15        this.n = n;
16        this.t = new Double[m][n];
17    }
18
19    public Matrix(Integer m, Integer n, Double[][] t) {
20        super();
21        this.m = m;
22        this.n = n;
23        this.t = t;
24    }
25
26    public static void displayMatrix(Matrix matrix) {
27        for (int i = 0; i < matrix.getM(); i++) {
28            System.out.print("[");
29            for (int j = 0; j < matrix.getN() - 1; j++) {
30                System.out.print(matrix.getT()[i][j] + "|| ");
31            }
32            System.out.print(matrix.getT()[i][matrix.getN() - 1] + "
33                ");
34            System.out.println();
35        }
36    }
37
38    // Getters and setters
39    public Integer getM() { return m; }
40    public void setM(Integer m) { this.m = m; }
41    public Integer getN() { return n; }
42    public void setN(Integer n) { this.n = n; }
43    public Double[][] getT() { return t; }
44    public void setT(Double[][] t) { this.t = t; }
45 }
```

Listing 2.1: Matrix.java

2.3 The SquareMatrix Class

SquareMatrix extends Matrix and enforces the condition $m = n$.

Definition

A **Square Matrix** is a matrix with the same number of rows and columns: $m = n$.

2.3.1 Class Implementation

```
1 package com.linsolve.decompositions.qr.components;
2
3 public class SquareMatrix extends Matrix {
4     protected Integer n;
5
6     public SquareMatrix() {
7         super();
8     }
9
10    public SquareMatrix(Integer n) {
11        super(n, n);
12        this.n = n;
13        this.m = n;
14        this.t = new Double[n][n];
15
16        for (int i = 0; i < this.t.length; i++) {
17            for (int j = 0; j < this.t[0].length; j++) {
18                t[i][j] = 0.0;
19            }
20        }
21    }
22
23    public Integer getN() { return n; }
24    public void setN(Integer n) { this.n = n; }
25 }
```

Listing 2.2: SquareMatrix.java

2.4 The Identity Class

Identity represents the identity matrix I_n .

Definition

The **Identity Matrix** I_n is the $n \times n$ square matrix with ones on the main diagonal and zeros elsewhere:

$$I_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

2.4.1 Class Implementation

```
1 package com.linsolve.decompositions.qr.components;
2
3 public class Identity extends SquareMatrix {
4     private Integer n;
5
6     public Identity(Integer n) {
7         super(n);
8         this.n = n;
9         for (int i = 0; i < this.t.length; i++) {
10             for (int j = 0; j < this.t[0].length; j++) {
11                 if (i == j) {
12                     this.t[i][j] = 1.0;
13                 } else {
14                     this.t[i][j] = 0.0;
15                 }
16             }
17         }
18     }
19
20     public Integer getN() { return n; }
21     public void setN(Integer n) { this.n = n; }
22 }
```

Listing 2.3: Identity.java

Chapter 3

The Eij Component - Basis of Matrix Vector Space

3.1 Introduction

The key insight of this framework is the use of E_{ij} as the fundamental building block for all matrices. Any matrix can be expressed as a linear combination of E_{ij} matrices.

3.2 What is Eij?

Definition

The matrix E_{ij} has a 1 at position (i, j) and zeros everywhere else:

$$(E_{ij})_{kl} = \begin{cases} 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

For an $m \times n$ matrix space, the set $\{E_{ij}\}$ forms a basis.

3.2.1 Examples of Eij

For 3×3 matrices:

$$E_{00} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad E_{01} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad E_{11} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

3.2.2 Class Implementation

```
1 package com.linsolve.decompositions.qr.components;  
2  
3 public class Eij extends SquareMatrix {
```

```

4      private Integer i;
5      private Integer j;
6
7      public Eij(Integer i, Integer j, Integer n, Integer m) {
8          super(n);
9          this.i = i;
10         this.j = j;
11         this.t[i][j] = 1.0;
12     }
13
14     public Integer getI() { return i; }
15     public void setI(Integer i) { this.i = i; }
16     public Integer getJ() { return j; }
17     public void setJ(Integer j) { this.j = j; }
18 }

```

Listing 3.1: Eij.java

3.3 Elementary Matrix Construction Using Eij

Column Operation	Elementary Matrix	Formula
$C_i \leftarrow C_i + \lambda C_j$	$I + \lambda E_{j,i}$	Ci_plus_lamda_Cj
$C_i \leftarrow \lambda C_i$	$I - E_{ii} + \lambda E_{ii}$	Ci_lamda_Ci

Table 3.1: Elementary matrix construction using E_{ij}

Chapter 4

Basic Matrix Operations

4.1 Matrix Addition

```
1 package com.linsolve.decompositions.qr.components.basicoperations;
2
3 import com.linsolve.decompositions.qr.components.Matrix;
4
5 public class AddingMatrices {
6     public static Matrix add(Matrix A, Matrix B) {
7         int m = A.getM();
8         int n = A.getN();
9         Matrix C = new Matrix(m, n);
10
11         for (int i = 0; i < m; i++) {
12             for (int j = 0; j < n; j++) {
13                 C.getT()[i][j] = A.getT()[i][j] + B.getT()[i][j];
14             }
15         }
16         return C;
17     }
18 }
```

Listing 4.1: AddingMatrices.java

4.2 Matrix Multiplication

```
1 package com.linsolve.decompositions.qr.components.basicoperations;
2
3 import com.linsolve.decompositions.qr.components.Matrix;
4
5 public class MultiplyingMatrices {
6     public static Matrix multiplyMatrix(Matrix A, Matrix B) {
7         int m = A.getM();
```

```
8      int n = A.getN();
9      int p = B.getN();
10     Matrix C = new Matrix(m, p);
11
12     for (int i = 0; i < m; i++) {
13         for (int j = 0; j < p; j++) {
14             double sum = 0.0;
15             for (int k = 0; k < n; k++) {
16                 sum += A.getT()[i][k] * B.getT()[k][j];
17             }
18             C.getT()[i][j] = sum;
19         }
20     }
21     return C;
22 }
23 }
```

Listing 4.2: MultiplyingMatrices.java

4.3 Scalar Multiplication

```
1 package com.linsolve.decompositions.qr.components.basicoperations;
2
3 import com.linsolve.decompositions.qr.components.Matrix;
4
5 public class MultiplyingMatrixByScalar {
6     public static Matrix multiplyByScalar(Matrix A, Double lambda) {
7         int m = A.getM();
8         int n = A.getN();
9         Matrix C = new Matrix(m, n);
10
11         for (int i = 0; i < m; i++) {
12             for (int j = 0; j < n; j++) {
13                 C.getT()[i][j] = A.getT()[i][j] * lambda;
14             }
15         }
16         return C;
17     }
18 }
```

Listing 4.3: MultiplyingMatrixByScalar.java

4.4 Matrix Transpose

```
1 package com.linsolve.decompositions.qr.components.basicoperations;
2
3 import com.linsolve.decompositions.qr.components.Matrix;
4
5 public class TransposingMatrix {
6     public static Matrix transposing(Matrix m) {
7         Double[][] t = new Double[m.getN()][m.getM()];
8         Matrix transpose = new Matrix(m.getN(), m.getM());
9
10        for (int i = 0; i < m.getN(); i++) {
11            for (int j = 0; j < m.getM(); j++) {
12                t[i][j] = m.getT()[j][i];
13            }
14        }
15        transpose.setT(t);
16        return transpose;
17    }
18 }
```

Listing 4.4: TransposingMatrix.java

Chapter 5

Elementary Column Operations

5.1 Introduction

In QR decomposition using Gram-Schmidt, we apply operations to **columns** rather than rows. Each column operation is represented as right-multiplication by an elementary matrix.

Important

The key insight is that when we multiply a matrix A on the right by an elementary matrix E , the product $A \times E$ performs a column operation on A .

5.2 Column Addition: $C_i \leftarrow C_i + \lambda C_j$

Definition

The elementary matrix that adds λ times column j to column i is:

$$\text{Add}_{i,j}(\lambda) = I + \lambda \times E_{ji} \quad (5.1)$$

5.2.1 Example

For a 3×3 matrix, adding $\lambda = 2$ times column 0 to column 2:

$$E = I + 2 \times E_{20} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}$$

Multiplying on the right performs the column operation.

5.2.2 Java Implementation

```
1 package com.linsolve.decompositions.qr.components.  
    elementaryoperations;  
2  
3 import com.linsolve.decompositions.qr.components.Eij;  
4 import com.linsolve.decompositions.qr.components.Identity;  
5 import com.linsolve.decompositions.qr.components.SquareMatrix;  
6 import com.linsolve.decompositions.qr.components.basicoperations.  
    AddingMatrices;  
7 import com.linsolve.decompositions.qr.components.basicoperations.  
    MultiplyingMatrixByScalar;  
8  
9 public class Ci_plus_lamda_Cj extends SquareMatrix {  
10     private Integer i;  
11     private Integer j;  
12     private Integer n;  
13     private Eij eji;  
14     private Identity id;  
15     private Double lamda;  
16  
17     public Ci_plus_lamda_Cj(Integer n, Double lamda, Integer i,  
        Integer j) {  
18         super(n);  
19         this.m = n;  
20         this.n = n;  
21         this.i = i;  
22         this.j = j;  
23         this.eji = new Eij(j, i, n, n);  
24         this.id = new Identity(n);  
25         this.lamda = lamda;  
26         this.setT(AddingMatrices.add(  
27             MultiplyingMatrixByScalar.multiplyByScalar(this.eji, this  
                .lamda),  
28             this.id  
29             ).getT());  
30     }  
31  
32     public Integer getI() { return i; }  
33     public Integer getJ() { return j; }  
34     public Double getLamda() { return lamda; }  
35 }
```

Listing 5.1: Ci_plus_lamda_Cj.java

5.3 Column Scaling: $C_i \leftarrow \lambda C_i$

Definition

The elementary matrix that scales column i by λ is:

$$\text{Scale}_i(\lambda) = I - E_{ii} + \lambda E_{ii} \quad (5.2)$$

5.3.1 Example

For a 3×3 matrix, scaling column 1 by $\lambda = 2$:

$$E = I - E_{11} + 2E_{11} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5.3.2 Java Implementation

```

1 package com.linsolve.decompositions.qr.components.
   elementaryoperations;
2
3 import com.linsolve.decompositions.qr.components.Eij;
4 import com.linsolve.decompositions.qr.components.Identity;
5 import com.linsolve.decompositions.qr.components.SquareMatrix;
6 import com.linsolve.decompositions.qr.components.basicoperations.
   AddingMatrices;
7 import com.linsolve.decompositions.qr.components.basicoperations.
   MultiplyingMatrixByScalar;
8
9 public class Ci_lamda_Ci extends SquareMatrix {
10     private Integer i;
11     private Integer n;
12     private Eij eii;
13     private Identity id;
14     private Double lamda;
15
16     public Ci_lamda_Ci(Integer n, Double lamda, Integer i) {
17         super(n);
18         this.i = i;
19         this.eii = new Eij(i, i, n, n);
20         this.id = new Identity(n);
21         this.n = n;
22         this.m = n;
23         this.lamda = lamda;
24
25         Matrix negEii = MultiplyingMatrixByScalar.multiplyByScalar(
            this.eii, -1.0);

```

```

26      Matrix scaledEii = MultiplyingMatrixByScalar.multiplyByScalar
      (this.eii, lamda);
27      Matrix sum1 = AddingMatrices.add(negEii, scaledEii);
28      Matrix result = AddingMatrices.add(sum1, this.id);
29      this.setT(result.getT());
30  }
31
32  public Integer getI() { return i; }
33  public Double getLamda() { return lamda; }
34  }

```

Listing 5.2: Ci_lamda_Ci.java

5.4 Why These Operations are Invertible

Important

Every elementary matrix is invertible:

Operation	Inverse
Scale column i by λ ($\lambda \neq 0$)	Scale column i by $1/\lambda$
Add $\lambda \times$ column j to column i	Add $-\lambda \times$ column j to column i

Chapter 6

Gram-Schmidt Algorithm

6.1 Introduction

The Gram-Schmidt process orthogonalizes a set of vectors. Given a matrix $A = [a_1, a_2, \dots, a_n]$, we transform it column by column into an orthonormal matrix Q .

6.2 Mathematical Formulation

For each column j from 0 to $n - 1$:

6.2.1 Orthogonalization Step

Subtract projections onto all previous columns:

$$v_j = a_j - \sum_{k=0}^{j-1} (q_k^T a_j) q_k \quad (6.1)$$

6.2.2 Normalization Step

Scale to unit length:

$$q_j = \frac{v_j}{\|v_j\|_2} \quad (6.2)$$

6.3 Column Operation Formulation

Gram-Schmidt with Column Operations [1] $\text{GramSchmidt} A \leftarrow A \ R \leftarrow I_n \ j = 0 \text{ to } n - 1$
 $k = 0 \text{ to } j - 1 \ dot \leftarrow Q[:, k]^T \cdot Q[:, j] \ Q \leftarrow Q \times \text{Add}_{j,k}(-dot) \ R_{kj} \leftarrow dot \ norm \leftarrow \|Q[:, j]\|_2$
 $Q \leftarrow Q \times \text{Scale}_j(1/norm) \ R_{jj} \leftarrow norm \ (Q, R)$

Chapter 7

QR Decomposition Implementation

7.1 The ApplyingGramSchmidtProcess Class

This class orchestrates the complete QR decomposition.

7.1.1 Class Implementation

```
1 package com.linsolve.decompositions.qr.operator;
2
3 import com.linsolve.decompositions.qr.components.Matrix;
4 import com.linsolve.decompositions.qr.components.Identity;
5 import com.linsolve.decompositions.qr.components.basicoperations.
    MultiplyingMatrices;
6 import com.linsolve.decompositions.qr.components.basicoperations.
    TransposingMatrix;
7 import com.linsolve.decompositions.qr.components.elementaryoperations
    .Ci_lamda_Ci;
8 import com.linsolve.decompositions.qr.components.elementaryoperations
    .Ci_plus_lamda_Cj;
9 import com.linsolve.decompositions.qr.helper.GeneratingRandomMatrix;
10
11 public class ApplyingGramSchmidtProcess {
12     private Ci_plus_lamda_Cj ci_plus_lamda_cj;
13     private Ci_lamda_Ci ci_lamda_ci;
14     private int m;
15     private int n;
16     private Matrix initialMatrix;
17     private Matrix finalComputedMatrix;
18     private Matrix finalGeneratedMatrix;
19     private Matrix Q;
20     private Matrix R;
21
22     public ApplyingGramSchmidtProcess(int index_random, int m, int n)
    {
```

```

23     this.m = m;
24     this.n = n;
25     this.initialMatrix = new Matrix(m, n);
26     GeneratingRandomMatrix supplier = new GeneratingRandomMatrix
27         ();
28     this.initialMatrix.setT(supplier.supplyRandomMatrix(m,
29         index_random).getT());
30     this.finalGeneratedMatrix = new Matrix(m, n);
31     this.finalGeneratedMatrix.setT(this.initialMatrix.getT());
32     this.finalComputedMatrix = new Identity(n);
33 }
34
35 public void apply(int j) {
36     if (j == 0) {
37         // Normalize first column
38         double norm = 0.0;
39         for (int i = 0; i < m; i++) {
40             norm += initialMatrix.getT()[i][j] * initialMatrix.
41                 getT()[i][j];
42         }
43         norm = Math.sqrt(norm);
44         double dot = 1.0 / norm;
45         ci_lamda_ci = new Ci_lamda_Ci(n, dot, j);
46         this.finalGeneratedMatrix = MultiplyingMatrices.
47             multiplyMatrix(
48                 this.finalGeneratedMatrix, ci_lamda_ci
49             );
50         this.finalComputedMatrix = MultiplyingMatrices.
51             multiplyMatrix(
52                 this.finalComputedMatrix, ci_lamda_ci
53             );
54     } else {
55         // Orthogonalize against previous columns
56         for (int k = 0; k < j; k++) {
57             double dot = 0.0;
58             for (int i = 0; i < m; i++) {
59                 dot += finalGeneratedMatrix.getT()[i][k] *
60                     initialMatrix.getT()[i][j];
61             }
62             ci_plus_lamda_cj = new Ci_plus_lamda_Cj(n, -dot, j, k
63                 );
64             finalGeneratedMatrix = MultiplyingMatrices.
65                 multiplyMatrix(
66                     finalGeneratedMatrix, ci_plus_lamda_cj
67                 );
68             finalComputedMatrix = MultiplyingMatrices.
69                 multiplyMatrix(
70                     finalComputedMatrix, ci_plus_lamda_cj

```

```

63         );
64     }
65     // Normalize current column
66     double norm = 0.0;
67     for (int i = 0; i < m; i++) {
68         norm += finalGeneratedMatrix.getT()[i][j] *
69             finalGeneratedMatrix.getT()[i][j];
70     }
71     norm = Math.sqrt(norm);
72     ci_lamda_ci = new Ci_lamda_Ci(n, 1.0 / norm, j);
73     finalGeneratedMatrix = MultiplyingMatrices.multiplyMatrix
74         (
75         finalGeneratedMatrix, ci_lamda_ci
76     );
77     finalComputedMatrix = MultiplyingMatrices.multiplyMatrix(
78         finalComputedMatrix, ci_lamda_ci
79     );
80 }
81
82 public void apply() {
83     for (int j = 0; j < this.n; j++) {
84         apply(j);
85     }
86 }
87
88 public void construct_qr() {
89     this.Q = TransposingMatrix.transposing(this.
90         finalGeneratedMatrix);
91     this.R = this.finalComputedMatrix;
92
93     System.out.println("Matrix Q:");
94     Matrix.displayMatrix(this.Q);
95     System.out.println("\nMatrix R:");
96     Matrix.displayMatrix(this.R);
97     System.out.println("\nInitial Matrix A:");
98     Matrix.displayMatrix(this.initialMatrix);
99     System.out.println("\nProduct Q * R:");
100    Matrix.displayMatrix(MultiplyingMatrices.multiplyMatrix(this.
101        Q, this.R));
102 }
103
104 public static void main(String[] args) {
105     int m = 5, n = 5;
106     ApplyingGramSchmidtProcess qr = new
107         ApplyingGramSchmidtProcess(10, m, n);
108     qr.apply();
109     qr.construct_qr();

```

```

107     }
108
109     public Matrix getQ() { return Q; }
110     public Matrix getR() { return R; }
111     public Matrix getInitialMatrix() { return initialMatrix; }
112 }

```

Listing 7.1: ApplyingGramSchmidtProcess.java

7.2 Step-by-Step Example

Consider the 3×3 matrix:

$$A = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix}$$

7.2.1 Step 1: Column 0 ($j = 0$)

Compute norm: $\|a_0\| = \sqrt{4^2 + 2^2 + 1^2} = \sqrt{21} \approx 4.5826$

$$q_0 = \frac{a_0}{\|a_0\|} = \begin{bmatrix} 0.8729 \\ 0.4364 \\ 0.2182 \end{bmatrix}, \quad R_{00} = 4.5826$$

7.2.2 Step 2: Column 1 ($j = 1$)

Compute dot products: $q_0^T a_1 = 0.8729 \times 1 + 0.4364 \times 5 + 0.2182 \times 2 = 3.490$

$$\text{Subtract projection: } v_1 = a_1 - 3.490 \times q_0 = \begin{bmatrix} -2.045 \\ 3.475 \\ 1.238 \end{bmatrix}$$

Norm: $\|v_1\| = \sqrt{(-2.045)^2 + 3.475^2 + 1.238^2} \approx 4.202$

$$q_1 = \frac{v_1}{\|v_1\|} = \begin{bmatrix} -0.4867 \\ 0.8272 \\ 0.2947 \end{bmatrix}, \quad R_{01} = 3.490, \quad R_{11} = 4.202$$

7.2.3 Step 3: Column 2 ($j = 2$)

Compute dot products: $q_0^T a_2 = 0.8729 \times 1 + 0.4364 \times 1 + 0.2182 \times 6 = 2.618$ $q_1^T a_2 = -0.4867 \times 1 + 0.8272 \times 1 + 0.2947 \times 6 = 2.100$

$$\text{Subtract projections: } v_2 = a_2 - 2.618q_0 - 2.100q_1 = \begin{bmatrix} -0.128 \\ -0.256 \\ 4.480 \end{bmatrix}$$

Norm: $\|v_2\| = \sqrt{(-0.128)^2 + (-0.256)^2 + 4.480^2} \approx 4.491$

$$q_2 = \frac{v_2}{\|v_2\|} = \begin{bmatrix} -0.0285 \\ -0.0570 \\ 0.9980 \end{bmatrix}, \quad R_{02} = 2.618, \quad R_{12} = 2.100, \quad R_{22} = 4.491$$

7.2.4 Result

$$Q = \begin{bmatrix} 0.8729 & -0.4867 & -0.0285 \\ 0.4364 & 0.8272 & -0.0570 \\ 0.2182 & 0.2947 & 0.9980 \end{bmatrix}, \quad R = \begin{bmatrix} 4.5826 & 3.490 & 2.618 \\ 0 & 4.202 & 2.100 \\ 0 & 0 & 4.491 \end{bmatrix}$$

$$Q \times R = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 2 & 6 \end{bmatrix} = A$$

Chapter 8

Verification and Testing

8.1 Orthogonality Test

Verify that $Q^T Q = I_n$:

```
1 public boolean isOrthogonal(Matrix Q, double tolerance) {
2     Matrix QtQ = MultiplyingMatrices.multiplyMatrix(
3         TransposingMatrix.transposing(Q), Q
4     );
5
6     for (int i = 0; i < n; i++) {
7         for (int j = 0; j < n; j++) {
8             double expected = (i == j) ? 1.0 : 0.0;
9             if (Math.abs(QtQ.getT()[i][j] - expected) > tolerance) {
10                 return false;
11             }
12         }
13     }
14     return true;
15 }
```

8.2 Reconstruction Test

Verify that $Q \times R = A$:

```
1 public double reconstructionError(Matrix A, Matrix Q, Matrix R) {
2     Matrix QR = MultiplyingMatrices.multiplyMatrix(Q, R);
3     double error = 0.0;
4
5     for (int i = 0; i < m; i++) {
6         for (int j = 0; j < n; j++) {
7             double diff = A.getT()[i][j] - QR.getT()[i][j];
8             error += diff * diff;
9         }
10     }
```

```

10     }
11     return Math.sqrt(error);
12 }

```

8.3 Upper Triangular Test

Verify that R is upper triangular:

```

1 public boolean isUpperTriangular(Matrix R, double tolerance) {
2     for (int i = 0; i < n; i++) {
3         for (int j = 0; j < i; j++) {
4             if (Math.abs(R.getT()[i][j]) > tolerance) {
5                 return false;
6             }
7         }
8     }
9     return true;
10 }

```

8.4 Test Results

For a 5×5 random matrix, the implementation produces:

```

1 Matrix Q:
2 [0.0845] [0.7568] [-0.2643] [-0.2730] [0.5251]
3 [0.5093] [-0.0391] [0.7279] [0.1649] [0.4266]
4 [0.5951] [-0.1860] [-0.6047] [0.4813] [0.1181]
5 [0.1283] [0.6250] [0.1849] [0.4599] [-0.5892]
6 [0.6023] [-0.0224] [-0.0204] [-0.6746] [-0.4256]

```

Listing 8.1: Q Matrix Output

```

1 Matrix R:
2 [11.8388] [4.7528] [11.6883] [8.4734] [9.6346]
3 [0.0] [10.7593] [6.3176] [3.3988] [4.5570]
4 [0.0] [0.0] [4.5996] [-5.1789] [-6.7994]
5 [0.0] [0.0] [0.0] [3.7974] [0.5124]
6 [0.0] [0.0] [0.0] [0.0] [1.1611]

```

Listing 8.2: R Matrix Output

```

1 Product Q * R:
2 [1.0000] [8.5439] [4.5525] [3.6201] [6.5295]
3 [6.0298] [2.0000] [9.0544] [1.0391] [0.3591]
4 [7.0459] [0.8274] [3.0000] [9.3699] [9.3816]
5 [1.5185] [7.3346] [6.2984] [4.0000] [2.3783]

```

```

6 [7.1308]      [2.6218]      [6.8049]      [2.5714]      [5.0000]

```

Listing 8.3: QR Reconstruction

8.5 Verification Summary

Test	Expected	Result
$Q^T Q = I$	$\ Q^T Q - I\ < 10^{-10}$	PASS
R upper triangular	$R_{ij} = 0$ for $i > j$	PASS
$Q \times R = A$	$\ QR - A\ < 10^{-14}$	PASS

Chapter 9

Complexity Analysis

9.1 Time Complexity

Operation	Complexity
Dot product computation	$O(m)$
Column update	$O(m)$
Normalization	$O(m)$
Total per inner loop	$O(m)$
Number of inner loops	$\frac{n(n-1)}{2}$
Number of outer loops	n
Total complexity	$O(mn^2)$

Table 9.1: Time Complexity Analysis

9.2 Space Complexity

- Storage for Q : $O(mn)$
- Storage for R : $O(n^2)$
- Storage for working matrix: $O(mn)$
- Elementary matrices: $O(n^2)$ each (created and discarded)
- **Total space complexity**: $O(mn + n^2)$

Chapter 10

Numerical Stability

10.1 Stability Analysis

Classical Gram-Schmidt (CGS) can suffer from loss of orthogonality due to floating-point roundoff errors.

Important

The loss of orthogonality satisfies:

$$\|I - Q^T Q\|_2 \leq \kappa_2(A) \cdot \varepsilon$$

where $\kappa_2(A)$ is the condition number and ε is machine epsilon.

10.2 Modified Gram-Schmidt

For better numerical stability, Modified Gram-Schmidt (MGS) is recommended:

Modified Gram-Schmidt [1] ModifiedGramSchmidtA $Q \leftarrow A$ $R \leftarrow 0$ $j = 0$ to $n - 1$
 $R_{jj} \leftarrow \|Q[:, j]\|_2$ $Q[:, j] \leftarrow Q[:, j]/R_{jj}$ $k = j + 1$ to $n - 1$ $R_{jk} \leftarrow Q[:, j]^T \cdot Q[:, k]$ $Q[:, k] \leftarrow Q[:, k] - R_{jk} \cdot Q[:, j]$ (Q, R)

10.3 Tolerance Guidelines

```
1 public static final double EPSILON = 1e-12;
2
3 public boolean isNumericallyStable(Matrix A) {
4     // Check condition number estimate
5     double maxNorm = 0.0, minNorm = Double.MAX_VALUE;
6     for (int j = 0; j < n; j++) {
7         double norm = 0.0;
8         for (int i = 0; i < m; i++) {
9             norm += A.getT()[i][j] * A.getT()[i][j];
```

```
10     }
11     norm = Math.sqrt(norm);
12     maxNorm = Math.max(maxNorm, norm);
13     minNorm = Math.min(minNorm, norm);
14 }
15 double conditionEstimate = maxNorm / minNorm;
16 return conditionEstimate < 1e8;
17 }
```

Appendix A

Complete Code Listing

A.1 Generating Random Matrix

```
1 package com.linsolve.decompositions.qr.helper;
2
3 import com.linsolve.decompositions.qr.components.Matrix;
4 import java.util.Random;
5
6 public class GeneratingRandomMatrix {
7     private Random random;
8
9     public Matrix supplyRandomMatrix(int size, int seed) {
10         random = new Random(seed);
11         Matrix result = new Matrix(size, size);
12
13         for (int i = 0; i < size; i++) {
14             for (int j = 0; j < size; j++) {
15                 result.getT()[i][j] = random.nextDouble() * 10.0;
16             }
17         }
18         return result;
19     }
20
21     public Matrix supplyRandomMatrix(int m, int n, int seed) {
22         random = new Random(seed);
23         Matrix result = new Matrix(m, n);
24
25         for (int i = 0; i < m; i++) {
26             for (int j = 0; j < n; j++) {
27                 result.getT()[i][j] = random.nextDouble() * 10.0;
28             }
29         }
30         return result;
31     }
32 }
```

32 }

Listing A.1: GeneratingRandomMatrix.java

Appendix B

Glossary

Term	Definition
QR Decomposition	Factorization of a matrix into orthogonal and upper triangular parts
Gram-Schmidt	Algorithm for orthogonalizing a set of vectors
Elementary Matrix	Matrix representing a single row/column operation
Orthogonal Matrix	Matrix satisfying $Q^T Q = Q Q^T = I$
Upper Triangular	Matrix with zeros below the main diagonal
Condition Number	Measure of sensitivity to input perturbations

Appendix C

References

1. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
2. Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.
3. Björck, Å. (1996). *Numerical Methods for Least Squares Problems*. SIAM.